



Université de
Sherbrooke

Université de Sherbrooke

SQL

Fonction, procédure et automatisme

SQL_06

Christina KHNAISSER (christina.khnaisser@usherbrooke.ca)

Luc LAVOIE (luc.lavoie@usherbrooke.ca)

(les auteurs sont cités en ordre alphabétique nominal)

Scriptorum/Scriptorum/SQL_06-Fn-Proc-Auto, version 1.0.0.a, en date du 2026-02-03

— document de travail, ne pas citer —

Plan

Introduction	3
1. Présentation	4
2. Les fonctions	5
3. Les procédures	16
4. Les automatismes	28
Conclusion	29
Références.	30
Définitions	31

Introduction

Le présent document a pour but de présenter une introduction au langage de définition de fonctions, de procédures et d'automatismes qu'on retrouve dans SQL.

1. Présentation

Sur la base de paramètres ou de l'état de la BD :

- Une fonction calcule une valeur suite à un appel.
- Une procédure change l'état de la BD suite à un appel.
- Un automatisme change l'état de la BD suite à un évènement.

2. Les fonctions

2.1. Syntaxe (PostgreSQL)

```
function_def ::=  
    CREATE [ OR REPLACE ] FUNCTION name ( [ argument [, ...] ] )  
        RETURNS ret_type { other_function_qual } ...  
        function_body  
  
argument ::=  
    [ arg_mode ] [ arg_name ] arg_type [ DEFAULT default_expr ]  
  
arg_mode ::=  
    IN | VARIADIC  
  
ret_type ::=  
    type_denotation | TABLE ( [ column_name column_type [, ...] ] )
```

```
function_body ::=  
    RETURN expression  
  | BEGIN ATOMIC [statement ;] ... END  
  | LANGUAGE lang_name AS string
```

2.2. Exemple (1)

Sous-type

Le numéro de téléphone ne comprend que les chiffres qui le composent (aucune marque d'édition telle que tirets, parenthèses, points, etc.). Les numéros des pays membres de l'ITU comprennent entre 8 et 13 chiffres. Les chiffres des numéros canadiens et états-uniens se répartissent ainsi : indicatif national : 1; indicatif régional : 3; équipement : 7.

```
create domain Telephone as
    varchar(13)
    constraint Telephone_ITU check(value similar to '[0-9]{8,13}');

create domain Telephone_CA as
    Telephone
    check (value similar to '[1][0-9]{10}');
```

Définition Indicatif Regional

```
create function indicatifRegional(t Telephone_CA)
  returns VARCHAR(3)
  return substr(t, 2, 3);
```

Utilisation Indicatif Regional

```
select IndicatifRegional('18191234567');
```


2.3. Exemple (2)

Identifier les activités du département d'informatique.

Définition

```
create or replace function
  estActiviteInfo(sigle SigleCours)
  returns boolean
  language SQL as
$$
select substr(sigle, 1, 3) in ('IFT', 'IGE', 'IGL', 'IMN')
$$;
```

Utilisation

```
select sigle, estActiviteInfo(sigle)
from activite;
```

2.4. Exemple (3)

Calculer la côte d'une note.

Définition SQL

```
create or replace function
  calculerCote(_note numeric(3))
  returns char(1)
  language SQL as
$$
select case
  when _note >= 90 then 'A'
  when _note between 80 and 89 then 'B'
  when _note between 60 and 79 then 'C'
  when _note between 40 and 59 then 'D'
  else 'E'
end
$$;
```

Définition plpgsql

```
create or replace function
  calculerCote(_note numeric(3))
  returns char(1)
  language plpgsql as
$$
declare
  cote char(1);
begin
  if _note >= 90 then cote := 'A';
  elsif _note between 80 and 89 then cote := 'B';
  elsif _note between 60 and 79 then cote := 'C';
  elsif _note between 40 and 59 then cote := 'D';
  else cote := 'E';
end if;
return cote;
end
$$;
```

Utilisation

```
with note_finale as (  
  select matricule,  
         activite,  
         sum(note) as noteF  
  from Resultat  
  group by matricule, activite  
)  
select matricule, activite, calculerCote(noteF) as cote  
from note_finale;
```

2.5. Exemple (4)

Calculer le bulletin d'une personne étudiante.

Définition SQL

```
create or replace function
calculerBulletin(_matricule universite.matricule)
returns Table(activite universite.siglecours,
              trimestre universite.trimestre,
              note universite.note,
              cote char(1))
language SQL as
$$
-- Hypothèse toutes les notes sont sur 100
with note_activite as
(
  select activite, trimestre, avg(note) as note
  from resultat
  where matricule = _matricule
  group by activite, trimestre
)
select activite, trimestre, note, calculerCote(note) cote
from note_activite;
$;
```

Utilisation

```
select *  
from calculerBulletin('15110132');
```

3. Les procédures

3.1. Syntaxe (PostgreSQL)

```
procedure_def ::=  
    CREATE [ OR REPLACE ] PROCEDURE name ( [ argument [, ...] ] )  
    { other_procedure_qual } ...  
    procedure_body
```

```
procedure_body ::=  
    BEGIN ATOMIC [statement ;] ... END  
    | LANGUAGE lang_name AS string
```


3.2. Exemple (1)

Définir une procédure pour la création de types d'évaluation.

Définition fonction

```
create or replace function
  deuxPremieresLettres(_description varchar)
  returns char(2)
  language SQL as
$$
  select case
    when count(mot) > 1 then upper(string_agg(left(mot, 1), ''))
    when count(mot) = 1 then upper(string_agg(left(mot, 2), ''))
  end
  from regexp_split_to_table(_description, '\s+') AS mot;
$;
```

Définition procédure

```
create or replace procedure
  typeEvaluation_ins(_description varchar)
  language SQL as
$$
insert into TypeEvaluation(code, description)
  values (deuxPremieresLettres(_description), _description);
$$;
```

3.3. Exemple (2)

Définir une procédure pour l'inscription des personnes étudiantes en informatique.

Modification du schéma

```
create table Inscription
(
  matricule Matricule not null,
  sigle      SigleCours not null,
  trimestre Trimestre not null,
  constraint Inscription_cc0 primary key (matricule, sigle, trimestre),
  constraint Inscription_cr0 foreign key (matricule) references Etudiant
(matricule),
  constraint Inscription_cr1 foreign key (sigle) references Activite (sigle)
);
```

Définition procédure SQL

```
create or replace procedure
    inscrireInfo(_matricule Matricule,
                _nom varchar(30),
                _adresse varchar(30),
                _trimestre Trimestre)
    language SQL as
$$
insert into Etudiant(matricule, nom, adresse)
    values (_matricule, _nom, _adresse);

insert into Inscription(matricule, sigle, trimestre)
    select _matricule, sigle, _trimestre
    from Activite
    where estActiviteInfo(sigle);
$$;
```

Définition procédure *plpgsql*

```
create or replace procedure
    inscrireInfo(_matricule Matricule,
                _nom varchar(30),
                _adresse varchar(30),
                _trimestre Trimestre)
    language plpgsql as
$$
declare
    found int;
begin
    select count(*) into found
    from Etudiant
    where matricule = _matricule;

    if not found = 0 then
        insert into Etudiant(matricule, nom, adresse)
            values (_matricule, _nom, _adresse);
        raise notice 'Ajout Etudiant %', _matricule;
    end if;

    insert into Inscription(matricule, sigle, trimestre)
```

```
select _matricule, sigle, _trimestre  
from Activite  
where estActiviteInfo(sigle);  
end  
$$;
```

Utilisation procédure

```
call InscrireInfo('20122334', 'Jeanne', 'Sherbrooke', '20261');
```

3.4. Exemple (3)

Archiver les activités qui ne sont plus enseignées depuis plus que deux ans.

Modification schéma

```
-- Active = 1, Archive = 0
alter table Activite
  add column statut char(1) not null default 1;
```


Définition archiver une activité

```
create or replace procedure
  archiver_activite(_sigle siglecours)
  language SQL as
$$
  update activite
    set statut = 0
    where sigle = _sigle;
$$;
```

Définition archiver des activités

```
create or replace procedure
  archiver_activite_2ans()
  language plpgsql as
$$
declare
  rec record;
begin
  for rec in select sigle
              from activite as r1
              where (extract('year' from current_date)::int - 2)
                  >= all (select substr(trimestre, 1, 4)::int
                        from Resultat as r2
                        where r1.sigle = r2.activite)

  loop
    call archiver_activite(rec.sigle);
  end loop;
end
$;
```

Utilisation

```
call archiver_activite_2ans();
```

4. Les automatismes

À venir...

Conclusion

Références

PG

[fr] <https://docs.postgresql.fr/16/index.html> (2024-04-01)

[en] <https://www.postgresql.org/docs/16/index.html> (2024-04-01)

Définitions

SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-02-10 17:42:21 UTC



Université de Sherbrooke